

学 号:	1023004637 1023004714 1023004539
------	--

武汉理工大学

软件工程实践（二）

学 院	计算机与人工智能学院
专 业	软件工程
班 级	软件工程 2302
姓 名	赵昊、程宇、葛竞航
编 号	Last-Occutism
指导教师	唐祖锴

2026 年 6 月 20 日

目录

1. 任务概述.....	1
2. 小组任务计划.....	1
2.1. 任务分析	2
2.2. 项目设计	2
2.3. 团队组成与分工	3
2.4. 软件工具与项目约束	3
3. 小组软件过程.....	4
3.1. 代码仓库	4
3.2. 软件版本计划	4
3.3. 开发分支模型与提交流程	4
3.4. 代码规范与检查流程	5
3.5. 项目测试计划与实施流程	5
3.6. 项目集成与发布规范	5
3.7.	错误!未定义书签。
4. 小组成员任务报告	6
4.1. 成员一任务分工及实施过程描述	6
4.2. 成员二任务分工及实施过程描述	6
4.3. 成员三任务分工及实施过程描述	7
4.4. 成员四任务分工及实施过程描述	错误!未定义书签。
5. 实施过程问题记录与分析.....	7
6. 任务总结.....	8
7. 参考文献.....	8

1. 任务概述

本项目是一款基于东方 Project 风格开发的 Unity 竖版弹幕射击游戏，仓库地址为 <https://github.com/wutkst/kai-fa-last-occultism.git>。项目当前 master 分支已归档到 v1.0.0 版本，提交 bleelec 标记为“feat: v1.0 正式版本验收归档”。项目以 Unity 2022.3 为主要开发环境，C# 为主要开发语言，结合 Unity 场景、动画控制器、预制体、材质、TextMesh Pro 字体资源、音频资源、JSON/CSV 配置文件与少量 Shader 资源，完成了从开始菜单、角色选择、难度选择、关卡战斗、暂停、对话、Boss 战、结算界面到后续场景切换的完整游戏流程。



项目核心内容包括玩家机体移动与攻击、灵梦和魔理沙不同机体的常规攻击与符卡表现、敌人波次生成、敌人弹幕系统、Boss 战系统、道具掉落与回收、分数和擦弹统计、背景滚动与伪 3D 视觉表现、音乐室、游戏说明、CSV 对话文本、JSON 场景重置配置等。代码中体现了 Unity 项目常见的软件设计思想，例如 Global_GameManager、Global_SceneManager、Global_AudioManager、Global_ObjectPool 等全局管理类使用单例模式，Global_GameManager 中通过 State 枚举管理 Menu、Gaming、Pause、Dialog、SpellCard、Frozen、FinalUI 等状态，Global_ObjectPool 使用字典、队列和活跃对象集合实现对象池，敌人生成和射击配置通过 EnemySpawnConfig、ShootMode、EnemyShoot 等脚本在 Inspector 中可视化配置。

本次实践任务的目的不只是完成一个游戏功能，而是围绕已有 Unity 项目进行拓展开发与 Git 小组协作。小组在开发中采用线下讨论分工、个人分支开发、远程提交、合并到 develop、再合并 release、测试后进入 master 的流程。项目最终不采用课程样例中的 Maven/JAR 打包方案，而是根据 Unity 项目的实际情况完成可执行文件打包，并在 GitHub 上使用 release/v1.0.0 分支与 v1.0.0 标签进行版本发布。对于课程要求中提到的可视化界面、世界观补充和功能扩展，本项目主要通过 Unity UI、TextMesh Pro、音乐室、对话 CSV、Boss 剧情资源、结算 UI 和场景资源完成；仓库中没有独立 Web 框架，因此报告不将其写作已实现内容。

2. 小组任务计划

本次实践的工作重点是将单人或初始状态的 Unity 项目推进为可由小组协同维护的完整游戏工程。计划阶段首先确定项目方向为东方风格弹幕射击游戏，随后按照游戏系统边界进行任务拆分：一部分负责全局框架、场景切换、状态管理、敌人波次、弹幕对象池和版本集成；一部分负责玩家攻击、

弹幕伤害、道具掉落、Boss 战资源和可执行文件打包；另一部分负责 UI、动画、场景表现、角色技能视觉效果和结算界面。开发过程以功能可运行为首要约束，以 master 分支稳定、develop 分支支持持续集成、个人 feature 分支保留个人工作量为协作约束。

2.1. 任务分析

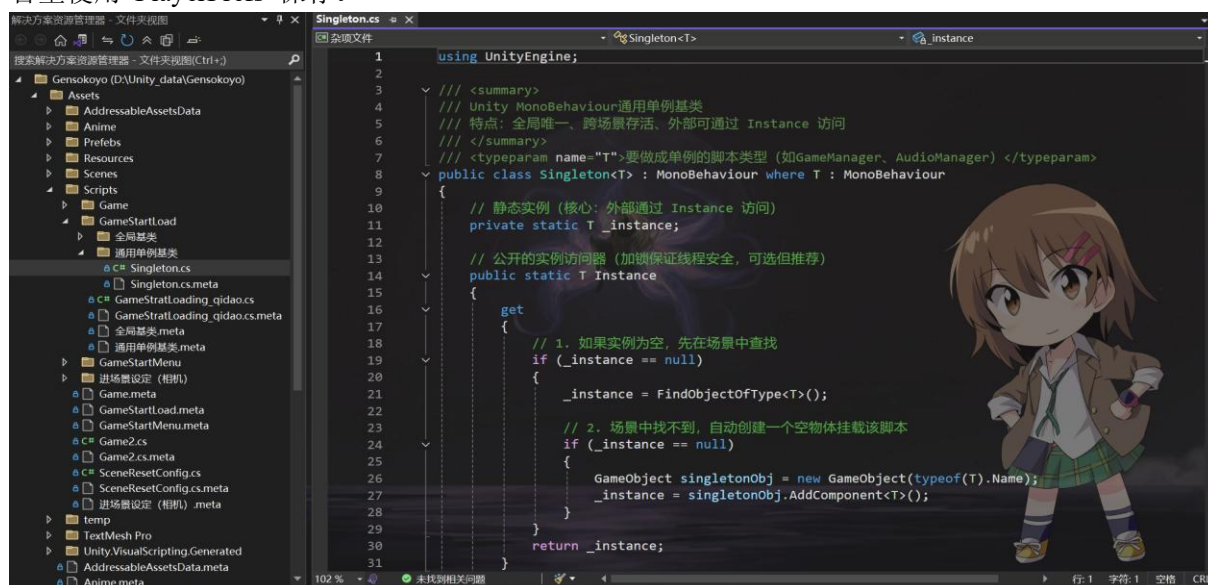
课程任务要求创建 3 至 4 人开发小组，围绕游戏项目进行功能扩充，使用 GitHub 进行多人协同开发，并在报告中说明分支模型、版本计划、代码提交、测试、集成和发布过程。结合本项目实际情况，小组没有基于 world-of-zuul 文本样例继续开发，而是选择 Unity 弹幕射击游戏作为实践对象。该选择提高了工程复杂度：Unity 工程不仅包含 C# 源码，还包含场景文件、动画控制器、预制体、图片、音频、字体、材质和 ProjectSettings 等资源文件，协作时更容易出现资源冲突、场景冲突和 meta 文件同步问题。

因此本次任务的重点包括：第一，保持 Unity 工程资源结构稳定，保证 Assets、Packages、ProjectSettings 能在不同成员环境下正常打开；第二，完成基础游戏循环，包括开始菜单、选人、选难度、战斗、暂停、死亡、重开、Boss 对话、Boss 战、结算与场景切换；第三，完成高频对象的性能优化，尤其是子弹、敌人、道具和特效对象池；第四，通过提交记录和分支保留每个成员的实际工作痕迹；第五，最终将 develop 的成果汇总到 release/v1.0.0，再经过测试修复后合并到 master，并打上 v1.0.0 标签。

2.2. 项目架构与功能设计

项目架构以 Unity 的 Assets 目录为核心。Assets/Scenes 中包含 GameStartLoading、GameStartMenu、Game1、Game2、GameOver 等场景；Assets/Scripts 按功能划分为 GameStartLoad、GameStartMenu、Game、进场景设定（相机）等目录；Assets/Prefabs 中保存敌人、子弹、道具、按钮等预制体；Assets/Anime 中保存角色、Boss、背景、符卡和结算动画；Assets/Resources/Touho 中保存 UI、Bgm、Boss、弹幕、对话、JSON、图片、字体等运行时资源。

核心框架方面，Singleton<T> 提供通用单例基类，Global_GameManager 保存角色、难度、生命、Bomb、Power、得分、擦弹、场景等级和游戏状态，并通过事件通知 UI 更新。Global_SceneManager 负责异步预加载场景、切换场景、读取 Resources/Touho/JSON 下的 JSON 文件进行场景重置，并在切换时清理对象池。Global_ObjectPool 使用对象池管理子弹、敌人、道具和特效，支持预加载、低余量扩容、对象回收和场景切换时清池。Global_AudioManager 负责 BGM、音效、淡出和音量持久化，音量使用 PlayerPrefs 保存。



游戏功能方面, CreateEnemy 按音乐播放时间和 EnemySpawnConfig 生成敌人, 支持 Path、Track、Flicker、Gravity 等移动模式以及二级移动模式; EnemyShoot 支持 diverge、random、sprial 等射击方式, 并可配置普通弹、追踪弹、拖尾弹、滞留弹、隐形弹等参数。玩家侧包含 PlayerAnime、PlayerCollision、Graze、ReimuShoot、MarisaShoot、MagicAttack、SpellCardEffect、ReimuNormal、ReimuSuper、MarisaNormal、MarisaSuper 等脚本, 实现移动、判定点、擦弹、受击、重生、常规攻击和符卡。Boss 模块包含 BossBase、BossAnime、BossBeheve、BossShootSystem、card1、card2、FinalCard、FreezeSystem、BossUI 等, 形成多阶段 Boss 战。UI 模块包含 UIManager、LeftLife、SpeelCard、ModeChange、GameOver、FinalUI、AboutDialog 等, 实现战斗信息、对话、结算分数、符卡获取状态和剧情文本显示。

2.3. 团队组成与分工

组长: 赵昊, 负责项目整体规划、任务分配、版本发布等工作。对应仓库中 feature/zhao hao/init-work 分支, 提交作者主要显示为 True ☆ Adiministrator、True ☆ Administrator 和 Ikaria。主要负责全局框架、状态管理、场景切换、对象池、敌人波次与弹幕配置、README、音乐室、对话系统、Boss 框架以及 release/master 集成。

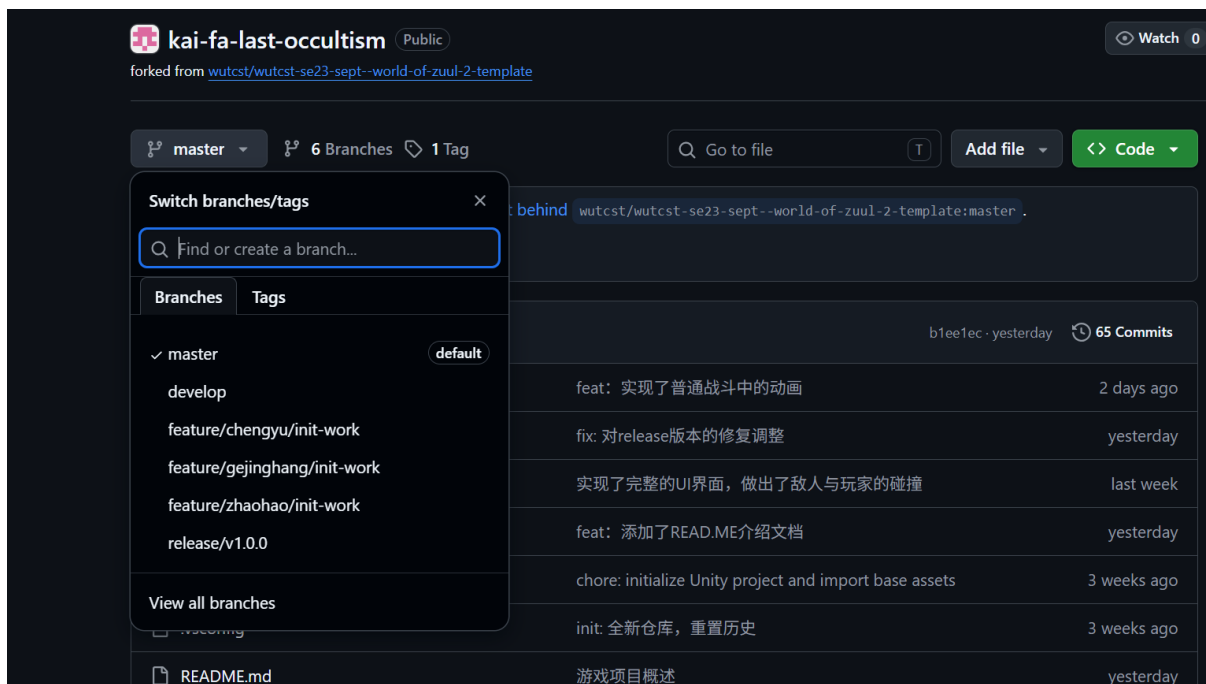
组员: 程宇, 负责游戏的核心逻辑开发、弹幕系统实现。对应仓库中 feature/chengyu/init-work 分支, 提交作者显示为 又一颗新星, 部分 Pull Request 合并记录显示为 QAQ-o。主要负责 Unity 项目基础资源、敌机弹幕资源与生成配置、玩家移动检测、重生动画、追踪子弹、子弹伤害、敌人生命值、掉落物生成、道具回收线逻辑、Boss 战资源和最终 exe 打包及路径问题修复。

组员: 葛竞航, 负责游戏的 UI 设计、动画制作、场景搭建等工作。对应仓库中 feature/gejinghang/init-work 分支, 提交作者显示为 erge1024。主要负责角色选择与难度选择 UI、完整游戏 UI、敌人与玩家碰撞、背景控制脚本和 Shader、背景滚动和伪 3D 效果、角色攻击表现、魔理沙常态与决死效果、机体技能动画、结算 UI 以及多处资源冲突和场景切换问题修复。

2.4. 软件工具与项目约束

项目主要工具为 Unity 2022.x、C#、Visual Studio Code、Git、GitHub、TextMesh Pro、Unity Animator、Unity Prefab、PlayerPrefs、JSON、CSV 和 GitHub Release。仓库中存在 Packages/manifest.json、ProjectSettings/ProjectVersion.txt、ProjectSettings/EditorBuildSettings.asset 等 Unity 工程文件, 说明项目依赖 Unity 包管理和编辑器配置。项目没有 .github 工作流目录, 因此没有形成完整的 GitHub Actions 自动化检查流程; 实际开发中主要通过本地 Unity 编辑器运行、场景测试、手动合并检查和最终打包验证保证质量。

项目约束包括: 所有成员必须在个人 feature 分支提交工作, 不能只在本地修改; develop 分支作为集成分支, master 保持稳定版本; Unity 资源修改必须同步 .meta 文件, 避免资源 GUID 丢失; 场景文件、预制体和 ProjectSettings 冲突需要谨慎处理; 提交说明尽量使用 feat、fix、chore 或中文功能描述, 便于后续报告和验收追踪; 最终发布必须从 release/v1.0.0 进入 master, 并以 v1.0.0 标签归档。报告整理阶段使用 AI 辅助阅读本地提交记录、归纳报告文本, 但所有项目功能描述均以当前 master 分支源代码、README 和 git log 为依据。



3. 小组软件过程

小组开发过程大体经历了初始化、功能扩展、系统集成、发布修复和验收归档五个阶段。初始化阶段在 2026-05-30 完成 Unity 工程创建和基础资源导入。功能扩展阶段集中在 2026-05-31 至 2026-06-18，三位成员分别在个人分支持续提交 UI、弹幕、对象池、敌人波次、道具、角色技能、Boss 和剧情功能。系统集成阶段多次将个人 feature 分支合并到 develop，并处理资源冲突、场景切换异常和对象池复用问题。发布阶段在 2026-06-19 将 develop 合并到 release/v1.0.0，经过修复后合并到 master，最终以 v1.0.0 标签归档。

3.1. 代码仓库

代码仓库为 <https://github.com/wutcst/kai-fa-last-occultism.git>。当前本地 master 分支与 origin/master 一致，最新提交为 b1ee1ec，提交说明为“feat: v1.0 正式版本验收归档”，并打有 v1.0.0 标签。远程分支包括 origin/develop、origin/feature/chengyu/init-work、origin/feature/gejinghang/init-work、origin/feature/zhaohao/init-work、origin/release/v1.0.0 和 origin/master。仓库根目录包含 README.md、Assets、Packages、ProjectSettings、.gitignore、.vsconfig 等 Unity 工程必需内容。README.md 已记录项目概述、技术栈、对象池、单例、状态机、贝塞尔曲线、Perlin 噪声、JSON/CSV、PlayerPrefs 等设计说明。

3.2. 软件版本计划

版本计划按 Unity 游戏功能成熟度划分。第一阶段为工程初始化版本，完成 Unity 项目创建、资源导入、开始菜单、加载场景和基础脚本。第二阶段为核心玩法版本，完成玩家移动、射击、碰撞、敌人生成、敌人弹幕、对象池和道具系统。第三阶段为表现增强版本，完成角色选择、难度选择、完整 UI、背景滚动、伪 3D 效果、角色动画、符卡动画、音频和暂停逻辑。第四阶段为 Boss 与剧情版本，完成对话系统、Boss 框架、Boss 弹幕、冰冻机制、关底结算 UI 和世界观文本补充。第五阶段为发布版本，修复打包路径、场景切换和资源问题，完成 exe 打包，并通过 release/v1.0.0 分支和 v1.0.0 标签发布。

3.3. 开发分支模型与提交流程

项目采用简化的 GitFlow 模型。master 分支只保存稳定验收版本；develop 分支作为日常集成分支；feature/zhaohao/init-work、feature/chengyu/init-work、feature/gejinghang/init-work 分别作为个人功能开

发分支；`release/v1.0.0` 作为最终发布候选分支。开发流程为：成员先在个人分支开发并提交，阶段性推送到远程；需要集成时由成员或组长发起 Pull Request 或直接合并到 `develop`；`develop` 中出现冲突或问题时由相关功能负责人线下沟通修复；发布前将 `develop` 合并到 `release/v1.0.0`；在 `release` 分支测试和修复后合并到 `master`；最后在 `master` 上打 `v1.0.0` 标签。

提交记录中可见多次合并过程，例如 2026-06-09、2026-06-12、2026-06-13、2026-06-19 多次将 `feature/chengyu/init-work` 通过 Pull Request 合并；2026-06-10、2026-06-12、2026-06-15、2026-06-16、2026-06-19 多次将 `feature/gejinghang/init-work` 合并；`feature/zhaohao/init-work` 则多次合并到 `develop`，并在 2026-06-19 进入 `release/v1.0.0` 和 `master`。开发中曾出现流程不规范、分支管理混乱和需要删除分支重置仓库的情况，后续通过固定个人分支和 `release` 分支才使流程稳定下来。

3.4. 代码规范与检查流程

项目提交说明逐步形成了 `feat`、`fix`、`chore` 与中文功能说明结合的格式，例如“`feat`: 添加敌人发射弹幕的攻击方式”“`fix`: 修复对象池重置时的问题”“`feat`: 成功打包成了 `exe`，修复了文件路径相关问题”。代码层面主要遵循 Unity C# 脚本结构，以 `MonoBehaviour` 生命周期函数、`Inspector` 暴露字段、`Header` 标注和模块化脚本组织为主。重要功能通过单独脚本拆分，例如 `Global_ObjectPool` 管理对象池，`CreateEnemy` 管理敌人波次，`EnemyShoot` 管理敌人射击，`FinalUI` 管理结算界面，`AboutDialog` 管理剧情对话。

检查流程主要是人工检查和本地 Unity 测试。由于仓库中没有 `.github/workflows` 目录，未实际配置 GitHub Actions 自动格式检查或自动测试，这是与课程理想要求存在差距的地方。实际开发中依靠成员本地打开 Unity 工程、运行 `GameStartMenu` 和 `Game1` 场景、检查 Console 报错、确认预制体引用、测试场景切换、处理冲突后再合并。后续如果继续完善，可以补充 Unity Test Framework、C# 静态分析或 GitHub Actions 构建检查。

3.5. 项目测试计划与实施流程

测试计划分为功能测试、集成测试和发布测试。功能测试关注单个模块是否可用，例如开始菜单按钮是否能切换，角色选择和难度选择是否能修改 `Global_GameManager` 状态，玩家射击是否正常，敌人是否按时间生成，子弹是否通过对象池复用，碰撞后生命和道具是否变化，暂停和返回主菜单是否恢复 `Time.timeScale`。集成测试关注模块之间的协作，例如 `GameStartLoading` 进入 `GameStartMenu`，`GameStartMenu` 进入 `Game1`，`Game1` 中触发对话和 `Boss`，`Boss` 战后进入 `FinalUI`，`FinalUI` 结算后进入 `Game2`。发布测试关注打包后的 `exe` 是否能正确读取 `Resources` 路径下的 JSON、CSV、音频和图片资源。

实施过程中，提交记录反映了多次测试后的修复：2026-06-03 修复场景切换逻辑异常和暂停界面返回主菜单时时间缩放未恢复的问题；2026-06-08 修复敌人移动模式、子弹类型枚举和对象池复用问题；2026-06-13 修复动画视觉问题并完善动画系统内部逻辑；2026-06-19 修复打包成 `exe` 后的文件路径相关问题。这些修复说明测试并非只在最后进行，而是在功能集成过程中持续发现并处理。

3.6. 项目集成与发布规范

项目集成以 `develop` 分支为中心。个人分支的功能完成后先合并 `develop`，解决冲突后再提交。`release/v1.0.0` 分支用于最终候选版本，2026-06-19 先将 `develop` 合并到 `release/v1.0.0`，再经过修复调整，最后由 `master` 接收正式版本归档。最终 `master` 最新提交 `bleelec` 标记为 `v1.0.0`，说明项目已经形成可验收版本。由于项目是 Unity 游戏，最终发布产物不是课程样例中的 `jar` 文件，而是 Unity 打包得到的 Windows 可执行文件，并结合 GitHub Release 功能进行发行。发布前重点检查打包路径、`Resources` 加载路径、场景配置、音频资源、JSON/CSV 资源和 `exe` 运行结果。



4. 小组成员任务报告

4.1. 【赵昊】任务分工及实施过程描述

在本项目中主要承担组长、架构整合和版本发布工作。开发早期完成仓库重置、Unity 工程基础结构搭建、开始菜单、加载场景和核心脚本的初始化。随后围绕全局框架持续推进，修复了全局场景单例在场景切换中的跨场景摄像头残留问题，并为全局游戏管理类加入公用对象和状态管理逻辑，完成 GameStartMenu 的按钮切换逻辑和开始菜单动画。该阶段确立了项目后续多人协作的基础：单例类统一保存全局状态，场景管理类处理加载和切换，菜单状态由按钮事件驱动，避免每个成员重复编写临时管理脚本。

核心玩法方面，主要负责敌人波次、敌人移动、敌人弹幕配置和对象池框架。提交记录显示，开发过程中陆续完成玩家机体弹幕射击功能、对象池初步生成销毁逻辑、游戏物体碰撞、道具吸引收集、敌人移动方式、贝塞尔曲线路径平滑、追踪、闪烁、下落、路径点移动以及敌人发射弹幕逻辑。后续又重构敌人波次设计代码，使敌人种类、射击类型、子弹类型等参数可以在 Unity Inspector 面板中配置，并修复对象池重置问题。这些工作对应当前代码中的 Global_GameManager、Global_SceneManager、Global_ObjectPool、CreateEnemy、EnemySpawnConfig、EnemyShoot 等模块。

后期重点转向剧情、Boss 框架和发布集成。开发过程中添加了游戏内数据的 JSON 重置、重新开始游戏功能、游戏说明文本、普通战斗动画、对话系统、Boss 战基本框架、README 项目介绍文档和音乐室子功能。协作流程上，个人分支 feature/zhaohao/init-work 多次合并到 develop，最终参与 develop、release/v1.0.0、master 的集成，并完成 v1.0 正式版本验收归档。作为组长，也承担了分支流程纠偏工作：早期分支和提交不够规范，经历过删除分支、重置仓库、重新约定个人分支与发布分支的过程，最终将成果稳定在 master 和 v1.0.0 标签上。

4.2. 【程宇】任务分工及实施过程描述

在本项目中主要承担核心逻辑、弹幕系统、资源配置和最终打包工作。开发初期完成 Unity 项目初始化、基础资源导入、自机移动检测、选人界面动画修复和自机虹人环特效。该阶段为玩家操作和角色表现打下基础，使后续弹幕、碰撞和技能系统可以围绕已有玩家对象继续扩展。在个人分支 feature/chengyu/init-work 上，也多次将 develop 合并回个人分支，保证自己的开发内容能及时跟上主线变化，减少多人并行开发时的冲突。

弹幕与战斗逻辑是主要工作重点。提交记录显示，该分支陆续添加敌人子弹资源和生成配置，完善敌机弹幕系统并更新场景资源，完成重生动画、子弹追踪效果、子弹伤害、敌人生命值、掉落物生成、敌人血量优化和道具回收线逻辑，并完成多种符卡攻击的资源调配。这些工作支撑了当前仓库

中的 Bullet、Enemy、Item、SpellEffect、PlayerCollision、Graze、AboutItem、CreateItem 等模块，使游戏从简单移动与射击扩展为具备伤害、掉落、回收、复活和符卡资源的完整战斗循环。

中后期工作转向 Boss 资源、关底流程和打包。开发过程中添加了上手项目所需的全部资源与基础配置，涉及 Boss 动画、Boss 子弹预制体、背景素材、音频、符卡素材、字体和 ProjectSettings 等 Unity 资源。最终还完成了 Unity 项目 exe 打包，并修复了打包后文件路径相关问题。由于 Unity 编辑器运行和 exe 运行对资源路径的要求不同，打包后需要重点验证 Resources、JSON、CSV、音频和场景配置能否正确加载。Git 协作方面，feature/chengyu/init-work 多次通过 Pull Request 合并到 develop，保留了个人开发痕迹，也保证了核心弹幕、资源配置和最终发布能力能进入正式版本

4.3. 【葛竞航】任务分工及实施过程描述

在本项目中主要承担 UI、动画、场景搭建和视觉表现工作。开发初期完成选角色和难度界面 UI，新增多组角色选择动画控制器、TextMesh Pro 字体材质和 GameStartMenu 场景资源。此后 UI 逐步扩展到战斗界面、暂停界面、结算界面和 Boss 相关界面。提交记录显示，后续实现了完整游戏 UI，并完成敌人与玩家碰撞相关内容，对应当前代码中的 UIManager、LeftLife、ModeChange、SpeelCard、PlayerCollision、PanDing、Graze 等脚本和多种道具预制体。该阶段不仅是界面绘制，也包含战斗数据与 UI 的联动，例如生命、Bomb、擦弹、得分、符卡等信息需要从全局状态中读取并显示。

视觉表现方面，完成背景控制脚本与 Shader，完善角色攻击表现和角色动画逻辑，实现游戏场景背景滚动与伪 3D 效果。当前仓库中可以看到 BGMove、BGImageScroll、ContinueBG、EvilShadow、UnlitBG.shader、ScrollBg.mat 等相关资源和脚本。随后又实现魔理沙机体的多种攻击方式，完成攻击特效表现和伤害判定，并修改攻击判定以匹配攻击特效。后续继续完成魔理沙常态与决死效果、机体技能动画设计，涉及 MarisaNormal、MarisaSuper、ReimuSuper、SpellCardEffect、ParticleConection、GunAnime、MagicAnime 等脚本和动画资源，使玩家技能具备动画、音效、材质和状态切换配合。

系统修复和集成方面，参与修复暂停界面返回主菜单时时间缩放未恢复的问题、场景切换过程中的逻辑异常、敌人移动模式、子弹类型枚举和对对象池复用相关问题。后期完成结算 UI，涉及 FinalUI、BossUI、AboutDialog、UIManager、BossBeheve、BossShootSystem 等脚本。FinalUI 中读取 CSV 文本，按顺序显示关底信息，计算分数、擦弹、Miss、耗时、符卡获取状态，并在结算后切换到 Game2。Git 协作方面，个人分支 feature/gejinghang/init-work 多次通过 Pull Request 合并到 develop，提交内容覆盖 UI、场景、动画、材质、预制体、碰撞和修复，最终随 release/v1.0.0 进入 master。

5. 实施过程问题记录与分析

第一个问题是开发流程早期不规范。项目一开始存在分支命名、提交归属和主线同步不够清晰的问题，甚至经历过删除分支和重置仓库。原因在于 Unity 工程体量较大，成员一开始更关注功能实现，没有先稳定分支模型。后续通过固定 master、develop、feature/*/init-work、release/v1.0.0 的分支结构，才使个人工作量和集成路线变得可追踪。

第二个问题是 Unity 资源冲突。与普通纯代码项目不同，Unity 场景、预制体、动画控制器、材质和 ProjectSettings 都可能被多人同时修改，合并时容易出现引用丢失、meta 文件冲突、场景对象状态错乱等问题。提交记录中多次出现“修复场景切换过程中的逻辑异常”“修复相关场景与资源冲突问题”“修复对象池复用相关问题”等说明。解决方式主要是减少多人同时修改同一场景，修改前线下沟通，合并后统一打开 Unity 检查场景和 Console 报错。

第三个问题是对象池和场景切换互相影响。弹幕游戏中子弹、敌人、道具数量多，如果直接 Instantiate 和 Destroy 会造成性能压力；但对象池跨场景存在时，又容易出现旧对象未回收、父对象失效、场

景切换后对象残留等问题。Global_ObjectPool 后续加入活跃对象集合、动态扩容、ClearAllPools、DisableAndRecycleAllActiveObjects 等逻辑，Global_SceneManager 在切换场景时调用对象池清理，缓解了该问题。

第四个人问题是运行环境与打包环境差异。Unity 编辑器中可运行的文件读取方式，在打包成 exe 后可能因为路径不同而失败。提交 7a0727d 明确修复了 exe 打包后的文件路径相关问题。后续项目将 JSON 和 CSV 放入 Resources/Touho 目录，通过 Resources.Load 读取，减少了编辑器路径和打包路径不一致带来的问题。

第五个人问题是自动化流程不足。课程任务建议使用 GitHub Actions 进行代码格式检查、测试和自动打包，但当前仓库没有 .github/workflows 目录，实际采用本地手动测试和人工合并检查。这说明小组已经完成了核心协作和发布流程，但 DevOps 自动化程度不足。后续改进方向是加入 Unity Test Framework、自动构建脚本、GitHub Actions 的 Unity Builder 或自定义构建命令，使 push 或 pull request 后能够自动检查编译和基本测试。

6. 任务总结

本次小组协同开发完成了一个基于 Unity 的东方风格竖版弹幕射击游戏，并在 master 分支形成 v1.0.0 验收版本。项目不仅包含开始菜单、选角选难度、战斗、暂停、重开、对话、Boss 战、结算和后续场景切换等完整流程，也实现了对象池、单例、状态机式状态管理、敌人波次配置、JSON 场景重置、CSV 对话文本、PlayerPrefs 本地保存、动画控制器、Shader 材质和多种音频资源整合。相较于课程样例项目，本项目在 Unity 资源协作、场景文件管理和发布打包方面复杂度更高，也更能体现多人协作中的工程问题。

从 Git 协作角度看，小组最终形成了个人 feature 分支开发、develop 集成、release/v1.0.0 测试发布、master 稳定归档的流程。虽然早期经历了分支不规范、删除分支、重置仓库等问题，但后续通过固定分支模型和远程合并保留了较完整的提交记录。每位成员的工作都能从 git log 中追踪：赵昊负责全局架构、状态和版本集成，程宇负责核心弹幕逻辑、资源配置和 exe 打包，葛竞航负责 UI、动画、视觉表现和结算界面。

本次实践的主要收获包括：第一，Unity 项目协作必须重视资源文件、meta 文件和场景冲突管理；第二，稳定的分支模型比临时合并更重要，尤其是多人并行开发时；第三，弹幕游戏需要对象池和状态管理，否则后期功能叠加会迅速失控；第四，发布不是简单点击打包，还要验证 Resources、JSON、CSV、音频和场景配置在 exe 中是否正常；第五，自动化测试和 CI 仍是项目短板，后续应补齐 GitHub Actions 或 Unity 自动构建流程。整体来看，本项目较好地完成了课程关于小组协同、功能扩展、版本管理和发布验收的核心要求。

7. 参考文献

[1]Unity 官方 Git 协作标准教程

Unity Technologies. Set up version control for your game project [EB/OL]. <https://learn.unity.com>, 2026.

[2] Saari M. Optimization for bullet hell games [D]. Kuopio: Savonia University of Applied Sciences, 2025.

（译文：萨阿里 M. 弹幕射击游戏性能优化 [学位论文]. 库奥皮奥：萨沃尼亚应用科学大学，2025.）

《软件工程实践（二）》成绩评定表

专业、班级	软件工程 2302	小组名称	Last-Occutism
代码仓库地址	https://github.com/wutest/kai-fa-last-occultism.git		
成员一姓名	赵昊	学号	1023004637
成员二姓名	程宇	学号	1023004714
成员三姓名	葛竞航	学号	1023004539
评价内容		满分	实得分
			得分
实践任务 完成情况	软件架构设计	25	
	软件版本计划与项目管理	25	
	软件分支管理	25	
	代码规范与代码评审	15	
	持续集成与部署	10	
实践任务评分		100	
实践报告 总评情况	学习态度与考勤	10	
	报告格式的规范性	20	
	报告的逻辑结构与语言表达	25	
	实践内容的正确性与合理性	40	
	文献引用及标注	5	
实践报告评分		100	
最终评定成绩（以优、良、中、及格、不及格评定）			

指导教师签字：

年 月 日